

DOCUMENTAZIONE FRAMEWORK MAP-REDUCE

Architettura generale

Il framework `libmr` implementa il modello MapReduce su singola macchina. Il programma utente non conosce i dettagli interni per via del tipo opaco `mr_t`.

Le fasi principali si distinguono in una fase di Map, dove ogni thread mapper crea coppie <token, valore> dopo aver ricevuto una riga di input, e di Reduce, nella quale i thread reducer applicano una specifica funzione alle coppie ricevute per produrre i risultati finali. Fra queste due fasi, le coppie con lo stesso token vengono raggruppate per migliorare l'efficienza dei reducer.

Tutte le funzioni implementate restituiscono -1 in caso di errore e 0 in caso di successo.

Tutti i thread sono gestiti unicamente da <threads.h> di C11 dunque con funzioni `thrd_create`, `thrd_join`, `mtx_t`, `cnd_t`.

Le code interne seguono una struttura che consiste in un buffer circolare `queue_item_t *buffer`, gli indici di testa e di coda, la dimensione della coda e la flag per la terminazione `shutdown` espresso come intero e i meccanismi di sincronizzazione `mtx_t mutex`, `cnd_t not_full` e `cnd_t not_empty`.

Fasi di esecuzione

1 - Inizializzazione

Nel programma utente (come in `examples/world_count.c`) viene eseguito il `main` che inizializza un'istanza di map-reduce con valori di default grazie alle funzioni `mr_attr_init`, che inizializza gli attributi del framework come numero di thread, grandezza della coda e hash, e le funzioni di setting `mr_attr_set...` che configurano il tutto con valori personalizzati dall'utente, rifiutando valori inaccettabili (ad esempio 0 thread o code di lunghezza 0). La funzione `mr_create` alloca memoria dinamicamente per la struttura di controllo del framework `struct_mr` copiandovi i parametri stabiliti e fornendo il puntatore all'istanza stessa.

2 - Avvio della pipeline

L'intera architettura del processo e la sua pipeline verrà fatta partire solo dopo che l'utente avrà chiamato `mr_start`, una funzione bloccante che ritornerà 0 solo al termine dell'esecuzione globale. Questa si occuperà di aprire un proprio file di log, creare le tre pipe UNIX per reindirizzare i dati in ingresso dal main ai mapper (`main_to_mapper`), dai mapper ai reducer (`mapper_to_reducer`) e poi di nuovo verso il main (`reducer_to_main`). Dopodiché, esegue le prime due fork per il processo mapper e il processo reducer con `pid_t m_pid = fork()` e `pid_t r_pid = fork()` duplicando i descrittori con `dup2` e connettendoli fra loro, avviando ciascuno con `run_mapper...` prima della creazione di qualunque thread. Ciascun figlio termina con `_exit(0)`. Le due dup specificano anche da dove leggere e dove scrivere i risultati (`STDIN` e `STDOUT`). Infine, il processo principale chiude tutto ciò che non è necessario grazie ai segnali di fine file `EOF`.

3 - Mapping

A seguito della chiamata `run_mapper_process`, eseguita subito dopo `fork`, viene inizializzata la coda per i dati dei mapper stabilendo anche la dimensione massima di quest'ultima. Successivamente vengono creati i threads, dopo avergli allocato memoria, implementando anche una struttura per passargli gli argomenti (`mapper_args_t m_args`). La funzione si occupa anche di estrarre i dati necessari dal file, come lunghezza e numero di righe.

Ora inizia il ciclo esecutivo vero e proprio dei mapper: viene letta ogni riga del file finché non si incontra `EOF` e per questa riga viene allocata memoria in base alla sua lunghezza e inserita in coda non dopo aver istanziato l'oggetto stesso

```
queue_item_t item = { f_name, l_num, line, l_len }.
```

Ogni riga logica viene serializzata come una sequenza di campi dopo aver rimosso `\n`:

```
[path_len: size_t ] → lunghezza del nome del file
[path: path_len byte ] → nome del path del file sorgente
[line_number: unsigned long ] → numero della riga nel file
[line_len: size_t ] → lunghezza del contenuto della riga (in byte)
[line: line_len byte ] → contenuto della riga
```

La fase di mapping avviene in parallelo su più thread di tipo worker e, in quanto la scrittura su pipeline condivisa richiede una sincronizzazione solida, la funzione di

callback `mapper_emit` acquisisce il mutex `mtx_t` prima di trasmettere, evitando di intaccare l'integrità del programma a causa di interleaving errati.

Terminato il ciclo while di lettura (causa fine del file) tutti gli worker vengono allertati (e risvegliati) che non ci sono più righe da leggere attraverso il flag `q.stop` e la variabile di condizione `cnd_broadcast`. Dopo aver atteso la terminazione di tutti i processi, viene liberata la memoria allocata e distrutto il mutex di controllo. Infine, dopo aver annotato gli eventi sul file di log il processo viene terminato con `exit(0)` che ritorna anche il segnale `E0F`.

I mapper worker operano convertendo l'argomento nella struttura specifica del mapper. Finché sono presenti elementi nella coda, prendono ogni elemento e lo copiano nella struttura `mr_file_line_t fl` per poi applicarvi la funzione mapper dell'utente tramite il passaggio della funzione `mapper_emit`. Gli worker si bloccano sulla variabile di condizione di `ts_queue_t` chiamata `not_full` quando la coda raggiunge la capacità massima mentre se è vuota attendono su `not_empty`. Il worker viene terminato quando il flag di stop viene posto a 1.

4 - Reducer

Dopo la chiamata di `run_reducer_process` vengono inizializzate le strutture dati necessarie per gli worker reducer come lo storage `st` per raccogliere le chiavi e i valori e la capacità totale di storage. Viene anche inizializzata una struttura temporanea `pkt` per leggere i pacchetti inviati dal mapper.

Dalla pipe del mapper ai reducer le coppie vengono passate dopo aver inviato un header binario contenente due valori a 4 byte:

`[token: token_len byte]` → stringa del token (con \0 rimosso)

`[value: val_size byte]` → sequenza di byte opachi

Dopo aver allocato memoria per il valore, questo viene letto e poi aggiunto alla struttura di accumulo `reducer_storage_t`. Dunque `add_storage` individua il gruppo `reducer_storage_t` del token se esiste già, altrimenti lo crea duplicandolo con `strdup` e stabilendo puntatore, allocando dinamicamente memoria per puntatori a valori e dimensioni ed eventualmente aumenta la capacità. Il valore del token viene copiato dunque in maniera opaca, ovvero senza far visualizzare all'utente tutti i dettagli, per mantenere più ordine e pulizia.

Adesso grazie a `qsort` i gruppi vengono ordinati, in maniera alfabetica come indicato dalla consegna per comodità, grazie alla funzione di callback implementata per gestire i gruppi di token `compare_groups`. Questo ordinamento garantisce il determinismo dell'intero programma.

In seguito, viene allocata la memoria per contenere i thread di tipo reducer che, dopo averli creati e atteso che tutti abbiano finito, viene liberata. Anche qui come per mapper, la funzione termina con `exit(0)`.

I reducer worker esaminano tutti i gruppi attualmente presenti nello storage alla ricerca di token di competenza del thread dopo aver applicato una funzione di hash per dividere il lavoro, come specificato da addendum. La funzione di hash specificata dall'utente si trova in `mr_attr_set_hash_function` e, se non specificata, si utilizza la funzione di default `djb2`, una delle funzioni più efficienti e semplici di C che utilizza come seme 5381. Se il risultato dell'applicazione della funzione di hash coincide con la specifica del thread (`if ((h % mr->attr.reducer_threads) == r_args->thread_id)`), questo alloca un array per la reimmissione del token grazie alla funzione callback `reducer_emit`.

Grazie alla chiamata della funzione `my_reducer`, che passa il reducer dell'utente come argomento di `mr_create`, si sommano tutte le occorrenze di una specifica parola e in seguito viene chiamata `reducer_emit`. Qui viene acquisito il mutex per evitare race conditions sulla pipe chiamata la funzione `written` con gli standard di output `STDOUT_FILENO` che restituisce in ordine l'istestazione con le lunghezze dei token e del risultato convertite in int, i byte della stringa del token e del risultato.

5 - Terminazione workers e fine del file

Le coppie sulla pipe che legge dai reducer e trasferisce sulla pipe principale del main hanno lo stesso formato delle precedenti.

Ogni reducer worker ritorna 0 e termina la sua esecuzione e dopo che tutti hanno terminato `run_reducer_process` libera lo spazio occupato dai thread e distrugge il mutex associato, terminando ancora con `exit(0)`. La chiusura manuale della funzione permette la chiusura di tutti i descrittori associati e soprattutto invia `E0F` sulla pipe di ritorno al main, sbloccandolo.

6 - Raccolta nel main

Ogni volta che il Reducer invia un risultato sulla pipe, il Main legge l'header, valida che le dimensioni non siano corrotte o fuori dai limiti ragionevoli, alloca la memoria necessaria e memorizza l'elemento all'interno di un array dinamico di strutture temporanee chiamato `recs`. Il ciclo while si interrompe istantaneamente quando viene segnalato `EOF` e, dato che i dati sulla pipe sono posti in maniera disordinata a seguito del diverso scheduling delle pipeline, viene invocata la funzione `qsort` per mantenere il determinismo del programma, in quanto non è garantito che l'invocazione parallela delle funzioni reducer produca sempre lo stesso ordinamento. La funzione di callback `compare_out_records` riordina lessicograficamente il tutto.

7 - Output

Una volta ottenuto l'array perfettamente ordinato, viene aperto il file destinazione output.mro con `open(output_path, O_WRONLY | O_CREAT | O_TRUNC, 0644)` sul quale verranno scritti tutti i record (`recs`) scritti nel formato prestabilito dalla consegna. Infine viene liberata la memoria allocata per i record di output e, per evitare che dei processi zombie restino in circolazione, `mr_start` si pone in attesa con `waitpid` sia di map che di reduce. Dunque il programma applicativo principale (in questo caso world_count.c) viene invocata `mr_destroy` e `mr_attr_destory` per pulire il tutto e chiudere prima di ritornare 0.

Il file di output è un file binario che contiene ogni record ordinato lessicograficamente per token nel formato:

`[token_len: int (4 byte)]` → lunghezza del token in byte (sempre senza \0)
`[token: token_len byte]` → stringa del token
`[result_len: int (4 byte)]` → lunghezza del risultato in byte
`[result: result_len byte]` → risultato opaco

I/O parziale

Per evitare casi di letture e scritture parziali sulla pipe, la comunicazione tra processi è veicolata dalle funzioni implementate `readn` e `writen` che forzano le operazioni all'interno di cicli while.

File di Log

Il file di log segue la struttura richiesta [timestamp] [processo(thread)] [evento] (messaggio). Grazie a `strftime` vengono segnalati adeguatamente i timestamp delle varie operazioni. La mutua esclusione in fase di scrittura dei file di log è garantita dalla struct `flock`.

Descrizione dei test

I test automatici realizzati in `tests/test_mr.c` sfruttano due funzioni semplici quali `basic_mapper` e `basic_reducer` integrate proprio nel file di test stesso. I test effettuati sono:

- `single_word` - file con un unico token, verifica del conteggio base.
- `repeated_word` - stesso token ripetuto più volte nello stesso file, verifica del raggruppamento.
- `multiline` - più file e più righe, verifica dell'ordine di lettura e dell'unione dei risultati.
- `no_newline_end` - ultima riga di un file priva del carattere terminatore '\n', verifica richiesta della consegna.
- `mixed_case` - token con maiuscole/minuscole diverse, verifica la normalizzazione dei caratteri.
- `empty_line` - righe vuote all'interno di un file, che non devono generare errori né coppie spurie.
- `empty_file` - file di lunghezza zero, che deve produrre zero righe elaborate senza bloccare la pipeline.
- `attr_invalid_values` - verifica che i setter di `mr_attr_t` rifiutino correttamente parametri non validi (es. 0 thread, coda di dimensione 0).
- `output_sorted` - verifica che il file di output sia ordinato correttamente.

Addendum

Scansione ricorsiva directory

`process_input_path` è stata estesa per le sottocartelle annidate sfruttando la funzione `collect_files` che distingue file regolari e le directory. I file regolari vengono

uplicati i path e inseriti nell'array dei file. Le directory vengono scansionate aggiungendole a `full_path`, un array sul quale verrà poi chiamato nuovamente `collect_files`. Dopo che tutti i file sono stati raccolti, vengono ordinati e scansionati uno per uno con `scan_and_send_file`.

Statistiche finali

A tempo di esecuzione, vengono raccolti i dati necessari per aggiornare le statistiche per poi essere inseriti in un file separato inizializzato come `mr_stats.txt`. Visto che Mapper e Reducer sono separati dal processo principale Main, le statistiche come il conteggio delle coppie o dei token vengono inviate anch'esse a pacchetti `stats_pkt` con il valore prefissato speciale -1 della lunghezza del token.

Hashing personalizzato

L'utente ha la possibilità di fornire una funzione di hashing deterministica che verrà settata tramite `mr_attr_set_hash_function`. Se non viene specificata, la funzione di default è djb2.

Elaborazioni indipendenti

Ogni istanza `mr_t` è allocata in maniera indipendente, con attributi appositamente settati. Il framework garantisce l'assenza di interferenze in caso di istanze indipendenti in sequenza.